

# Unit: Searching Techniques

## Learning Outcomes

After completing this unit, students will be able to:

- Define searching and explain its importance.
  - Differentiate between Linear Search and Binary Search.
  - Analyze the time and space complexity of searching algorithms.
  - Write algorithms and C programs for searching.
  - Compare searching techniques based on efficiency.
- 

## 1. Introduction to Searching

### Definition

**Searching** is the process of finding the location (index) of a particular element (called the **key**) in a collection of data.

For example,

Array:

```
Index : 0  1  2  3  4
Value : 12 25 37 48 59
```

Searching for **37** returns **Index = 2**.

Searching is one of the most common operations performed on data structures.

---

### Applications of Searching

- Student record management
  - Employee database
  - Contact list
  - Banking systems
  - Search engines
  - Library management
  - Online shopping
-

## 2. Types of Searching

Searching techniques are mainly divided into two categories.

| Technique     | Data Requirement   |
|---------------|--------------------|
| Linear Search | Unsorted or Sorted |
| Binary Search | Sorted Only        |

## 3. Linear Search

### Definition

Linear Search (Sequential Search) checks every element one by one until the desired element is found or the list ends.

### Working

Suppose

Array

15 23 8 41 56 19

Search = 41

Comparison sequence

15 x

23 x

8 x

41 ✓

Element found after 4 comparisons.

### Algorithm

LinearSearch(A, n, key)

1. Start

```
2. for i = 0 to n-1
3.     if A[i] == key
4.         return i
5. return -1
6. Stop
```

---

## Example

Array

10 22 35 48 50

Key = 35

## Comparisons

10 ✗

22 ✗

35 ✓

## Result

Position = 2

---

## C Program: Linear Search

```
C

#include <stdio.h>

int linearSearch(int arr[], int n, int key)
{
    int i;

    for(i=0;i<n;i++)
    {
        if(arr[i]==key)
            return i;
    }

    return -1;
}
```

```
int main()
{
    int arr[]={10,20,30,40,50};
    int n=5;
    int key=30;

    int pos=linearSearch(arr,n,key);

    if(pos==-1)
        printf("Element not found");
    else
        printf("Element found at index %d",pos);

    return 0;
}
```

---

## Output

Element found at index 2

---

## Linear Search Analysis

Suppose there are **n elements**.

### Best Case

Element found in first position.

### Example

Key = 10

10 20 30 40

### Comparisons

1

### Time Complexity

$O(1)$

---

## Worst Case

Element is last or absent.

Example

10 20 30 40 50

Search = 50

Comparisons

5

Time Complexity

$O(n)$

---

## Average Case

Element may occur anywhere.

Average comparisons

$(n+1)/2$

Time Complexity

$O(n)$

---

## Space Complexity

Only one extra variable is used.

$O(1)$

---

## Advantages of Linear Search

- Very simple
- Works on unsorted arrays

- No preprocessing required
  - Suitable for small datasets
- 

## Disadvantages

- Slow for large datasets
  - Maximum comparisons can be  $n$
- 

## 4. Binary Search

### Definition

Binary Search repeatedly divides the search space into two halves.

**Requirement:** The array **must be sorted**.

---

### Working Principle

Given

```
10 20 30 40 50 60 70
```

Search

```
60
```

### Step 1

```
Low=0
```

```
High=6
```

```
Mid=(0+6)/2=3
```

```
40
```

Since

```
60 > 40
```

Search right half.

## Step 2

50 60 70

Mid=5

60

Found.

---

## Binary Search Algorithm

```
BinarySearch(A, low, high, key)
```

```
while(low <= high)
```

```
    mid = (low + high)/2
```

```
    if(A[mid] == key)
        return mid
```

```
    else if(key < A[mid])
        high = mid-1
```

```
    else
        low = mid+1
```

```
return -1
```

---

## Example

Array

5 10 15 20 25 30 35

Search

25

## Iteration 1

Mid=3

20

Search right.

---

## Iteration 2

25 30 35

Mid=5

30

Search left.

---

## Iteration 3

25

Found

---

# C Program: Binary Search

C

```
#include <stdio.h>
```

```
int binarySearch(int arr[], int n, int key)
```

```
{
```

```
    int low=0;
```

```
    int high=n-1;
```

```
    while(low<=high)
```

```
    {
```

```
        int mid=(low+high)/2;
```

```
        if(arr[mid]==key)
```

```
            return mid;
```

```
        else if(key<arr[mid])
```

```
            high=mid-1;
```

```
        else
            low=mid+1;
    }

    return -1;
}

int main()
{
    int arr[]={10,20,30,40,50,60,70};
    int n=7;
    int key=60;

    int pos=binarySearch(arr,n,key);

    if(pos!=-1)
        printf("Element not found");
    else
        printf("Element found at index %d",pos);

    return 0;
}
```

---

## Output

Element found at index 5

---

## Binary Search Analysis

Suppose there are **n elements**.

### Best Case

Middle element is searched.

$O(1)$

Only one comparison.

---

### Worst Case

Search continues until one element remains.

Each iteration reduces the array size by half.

### Example

|     |                 |                   |  |                 |                   |  |                 |
|-----|-----------------|-------------------|--|-----------------|-------------------|--|-----------------|
| n   |                 |                   |  |                 |                   |  |                 |
| ↓   | Array<br>Len= n | Compari-<br>sions |  | Array<br>Len= n | Compari-<br>sions |  | Array<br>Len= n |
|     | 64              |                   |  | 256             |                   |  | 99              |
| n/2 | 32              | 1                 |  | 128             | 1                 |  | 49              |
|     | 16              | 2                 |  | 64              | 2                 |  | 24              |
| ↓   | 8               | 3                 |  | 32              | 3                 |  | 12              |
|     | 4               | 4                 |  | 16              | 4                 |  | 6               |
| n/4 | 2               | 5                 |  | 8               | 5                 |  | 3               |
| ↓   | 1               | 6                 |  | 4               | 6                 |  | 1               |
|     |                 |                   |  | 2               | 7                 |  |                 |
| n/8 |                 |                   |  | 1               | 8                 |  |                 |
| ↓   |                 |                   |  |                 |                   |  |                 |
| ... |                 |                   |  |                 |                   |  |                 |
| ↓   |                 |                   |  |                 |                   |  |                 |
| 1   |                 |                   |  |                 |                   |  |                 |

Number of iterations

$$2^k = n$$

$$k = \log_2 n$$

Hence

$$\text{Time Complexity} = O(\log n)$$

### Average Case

$$O(\log n)$$

### Space Complexity

Iterative Binary Search

$O(1)$

Recursive Binary Search

$O(\log n)$

(due to recursion stack)

---

## Why Binary Search is Faster?

Suppose

$n = 1,000,000$

Linear Search

Worst comparisons

1,000,000

Binary Search

$\log_2(1000000)$

$\approx 20$  comparisons

Huge performance improvement.

---

## Complexity Analysis

| Algorithm     | Best   | Average     | Worst       | Space              |
|---------------|--------|-------------|-------------|--------------------|
| Linear Search | $O(1)$ | $O(n)$      | $O(n)$      | $O(1)$             |
| Binary Search | $O(1)$ | $O(\log n)$ | $O(\log n)$ | $O(1)$ (iterative) |

---

## Linear Search vs Binary Search

| Feature          | Linear Search   | Binary Search         |
|------------------|-----------------|-----------------------|
| Data Requirement | Sorted/Unsorted | Sorted only           |
| Method           | Sequential      | Divide and Conquer    |
| Best Case        | $O(1)$          | $O(1)$                |
| Average Case     | $O(n)$          | $O(\log n)$           |
| Worst Case       | $O(n)$          | $O(\log n)$           |
| Space Complexity | $O(1)$          | $O(1)$ (Iterative)    |
| Suitable For     | Small datasets  | Large sorted datasets |
| Implementation   | Easy            | Slightly more complex |

## Dry Run Example

### Linear Search

Array

11 22 33 44 55

Search = 44

| Comparison | Element | Result |
|------------|---------|--------|
| 1          | 11      | No     |
| 2          | 22      | No     |
| 3          | 33      | No     |
| 4          | 44      | Found  |

Comparisons = **4**

### Binary Search

## Array

11 22 33 44 55 66 77

Search = 55

| Step | Low | High | Mid | Value | Action       |
|------|-----|------|-----|-------|--------------|
| 1    | 0   | 6    | 3   | 44    | Search Right |
| 2    | 4   | 6    | 5   | 66    | Search Left  |
| 3    | 4   | 4    | 4   | 55    | Found        |

Comparisons = 3

## Viva Questions

1. What is searching?
2. What is the difference between Linear Search and Binary Search?
3. Why must an array be sorted before Binary Search?
4. Which searching algorithm works on unsorted arrays?
5. What is the worst-case time complexity of Linear Search?
6. What is the average-case complexity of Binary Search?
7. Which algorithm uses the Divide and Conquer technique?
8. What is the space complexity of iterative Binary Search?
9. When is Linear Search preferred over Binary Search?
10. Can Binary Search be applied to a linked list efficiently? Why or why not?

## Exam Tips

- **Linear Search:** Checks elements one by one. Works on any array. Time complexity:  **$O(n)$** .
- **Binary Search:** Requires a **sorted array**. Repeatedly halves the search space using the middle element. Time complexity:  **$O(\log n)$** .
- For **small or unsorted datasets**, Linear Search is usually appropriate.
- For **large sorted datasets**, Binary Search is much faster and more efficient.

